

Adaptive Independent Evidence Verification Engine (AIEVE)

Revised Architecture, Gap Analysis, and Publication-Ready Specification

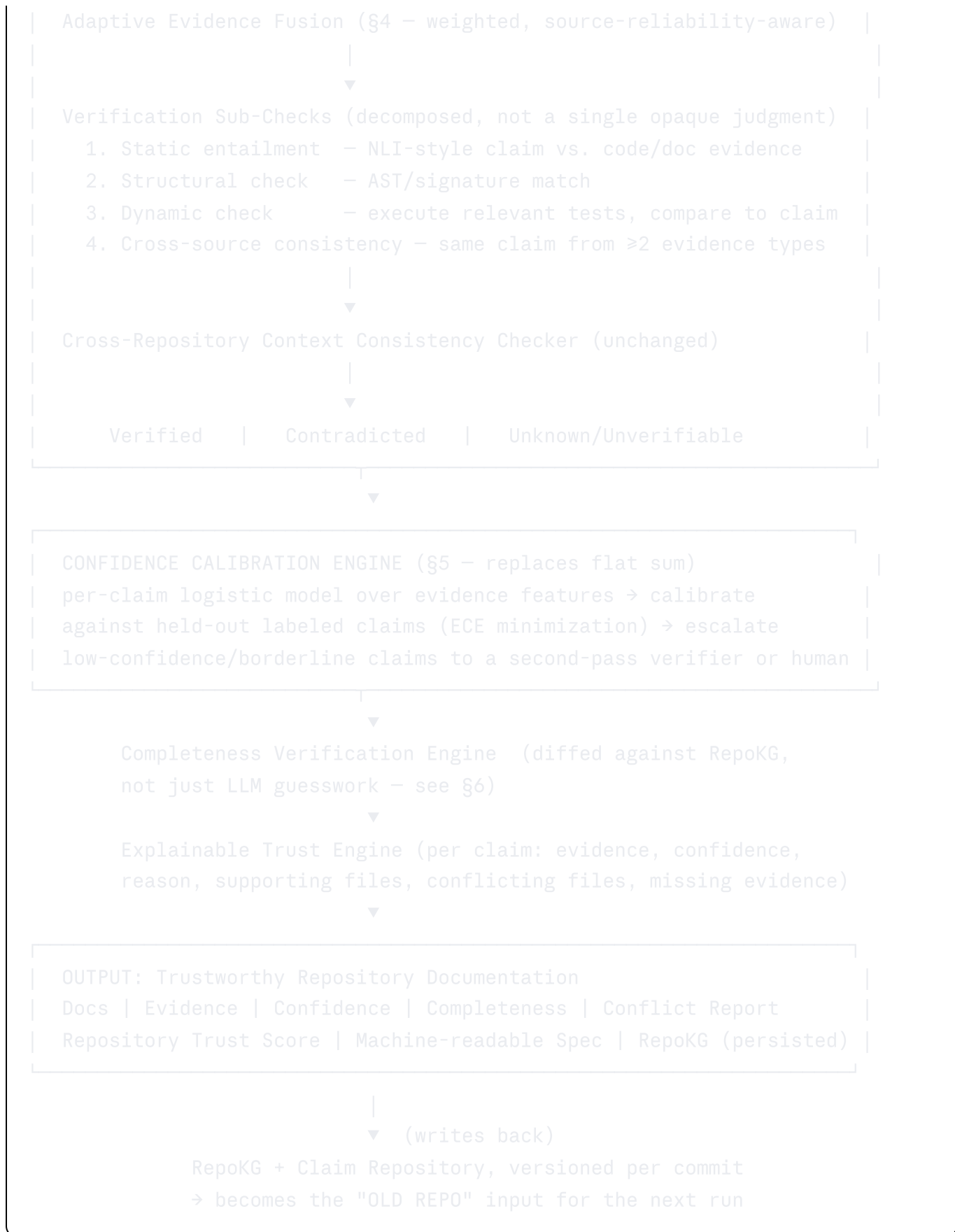
1. Positioning Against Recent Work

System	Core idea	Known limitation
RepoAgent (Luo et al., 2024)	Bidirectional reference graph + template docs, ordered by physical file structure	No semantic modularity; injects full caller/callee source into prompts (token blowup); only file-level change detection
CodeWiki (Makharev & Ivanov, 2025)	Retrieval-augmented, hierarchical decomposition by token/size limits	Decomposition ignores functional cohesion; bottom-up synthesis forces parent nodes to absorb full child source (context stacking); no incremental update support at all
RepoDoc (Xu et al., 2026)	Repository Knowledge Graph (RepoKG) as semantic backbone; Semantic Impact Propagation for incremental updates	Optimizes documentation <i>generation quality/cost</i> , not <i>trustworthiness</i> — no claim verification, no confidence scores, no contradiction detection
GraphCoder / RepoGraph / CodexGraph / KGCompass / LocAgent	Structural code graphs for retrieval-augmented completion/repair	Retrieval-accuracy focused; not documentation- or trust-focused
FActScore-style claim verification / VERDI / Atomic Calibration	Atomic claim decomposition, evidence-grounded verification, per-claim calibration	Built for open-domain NLG/QA, not adapted to source-code multi-evidence-type fusion

Your genuine white space: none of the documentation-generation systems above verify their own output against evidence or report a calibrated trust score. None of the claim-verification/calibration systems above are adapted to repositories (code + tests + commits + issues as heterogeneous evidence types). AIEVE sits exactly in that gap — *if* it (a) reuses a proper KG backbone instead of ad hoc multi-source collection, (b) adds the missing incremental-update path, and (c) turns the confidence "engine" into an actual calibrated model instead of a summed heuristic. Those are the three changes below.

2. Revised End-to-End Architecture





The key structural fix: **the output writes back into the RepoKG**, so the system has state across runs. That single feedback edge is what turns this from a one-shot pipeline into something that can handle repository evolution, which is the exact capability CodeWiki lacks entirely and RepoAgent only does at file granularity.

3. RepoKG Schema (fixes the "underspecified graph" gap)

Entity types

- `CodeEntity` — function/class/interface/module; stores source, signature, file path, visibility
- `ConceptEntity` — business/domain concept extracted by LLM, not explicit in code structure
- `ClaimEntity` — **your addition**: a generated claim, with fields `{text, source_agent, commit_hash, status, confidence, evidence_refs[]}`
- `EvidenceEntity` — **your addition**: a retrieved evidence unit (test result, commit, issue, CI log) with `{type, source_ref, retrieved_at, reliability_prior}`

Relationship types: `calls`, `implements`, `extends`, `imports`, `contains` (structural — extracted without LLM, cheap); `semantic-impact` (code→concept, LLM-derived); `describes` (doc/claim → code); `supports` / `contradicts` (evidence → claim, **your addition** — this is what makes the graph queryable for conflict detection instead of running conflict detection as a separate ungrounded step).

Construction is two-pass, same as RepoDoc: **structural relationships first** (AST/tree-sitter parsing, no LLM cost), **semantic relationships second** (LLM-based, only on the structural skeleton, so the LLM never has to infer structure it could get for free).

4. Adaptive Evidence Fusion — formalized

Instead of a named box with no math, define fusion as a weighted combination over evidence sources, where weights are **learned/tuned per source type**, not fixed:

```
fusion_score(claim c) =  $\sum_i w_i \cdot \text{sim}(c, e_i) \cdot \text{reliability}(\text{source\_type}(e_i))$ 
```

where:

- `e_i` = i-th retrieved evidence unit for claim c
- `sim(c, e_i)` = entailment/semantic-similarity score (NLI model or LLM judge, constrained to output a probability, not free text)
- `reliability(source_type)` = prior weight per evidence type, e.g.:

test execution result	→ 0.95	(ground truth, code actually ran)
CI/build config	→ 0.85	
committed source code	→ 0.80	
commit message	→ 0.55	(author-asserted, unverified)
issue/PR discussion	→ 0.45	(opinion/discussion, weakest)
- `w_i` = adaptive weight, reduced when `source_type(e_i)` has disagreed with test-execution ground truth historically for this repo (this is the "adaptive" part — weights update per-repository)

This is "adaptive" in the sense the name promises: reliability priors start from the fixed values above but get down-weighted for a specific repository/evidence-type pair if that combination has produced contradicted claims in past verification passes (stored back in RepoKG as feedback).

5. Confidence Calibration — formalized (this is your weakest current box)

Do **not** sum unrelated dimensions into one number and call it confidence — that has no calibration guarantee and current research shows verbalized LLM confidence is reliably overconfident on exactly this kind of aggregation. Instead:

1. **Feature vector per claim:** $[fusion_score, num_independent_sources_agreeing, static_entailment_prob, dynamic_test_pass_rate, historical_accuracy_of_this_claim_type]$
 2. **Calibration model:** a small logistic regression / gradient-boosted model (not the LLM itself) trained on a held-out set of claims you've manually labeled verified/contradicted, mapping the feature vector $\rightarrow P(\text{claim is correct})$. This is standard practice in the calibration literature (Platt scaling / isotonic regression / ECE minimization) and gives you an actual probability instead of a heuristic sum.
 3. **Escalation policy:** claims with $0.4 < P < 0.7$ (the ambiguous band) get routed to a second, independent verification pass or flagged for human review rather than silently assigned a mid-confidence score — this "trust or escalate" pattern is explicitly supported by recent LLM-judge calibration work and gives you a defensible way to bound false-confidence risk in the paper.
 4. **Report Expected Calibration Error (ECE)** in your evaluation — this is the metric reviewers will expect if you claim "calibrated confidence," and none of RepoAgent/CodeWiki/RepoDoc report anything like it, so it's a clean differentiator.
-

6. Completeness Verification — made concrete

Replace "Missing Modules / Missing APIs / ..." as vague categories with a diff against the RepoKG itself, mirroring RepoDoc's `Coverage` metric:

```
coverage = |{ CodeEntity : ∃ ClaimEntity describing it }| / |{ all public CodeEntity }|
```

Run this per category (modules, public APIs, business rules extracted vs. concepts in RepoKG, config keys read by code vs. documented, dependencies in manifest vs. documented, security-relevant entities e.g. auth/crypto calls vs. documented). Anything under threshold gets queued back into the multi-agent claim generation stage — this closes the loop between "completeness" and "generation" that your current diagram treats as one-directional.

7. Old-Repo vs. New-Repo Handling (the missing piece)

New repo (first run): full RepoKG construction → full clustering → full claim generation → full verification. Cache everything; this is your expensive path, run once.

Old repo (subsequent runs, given a commit diff):

1. **Change Detection** — AST diff between base commit and HEAD; classify each change (signature change, body-only change, addition, deletion, rename).
2. **Semantic Impact Propagation** — bidirectional BFS/queue traversal of the RepoKG from changed entities, following `calls`/`implements`/`semantic-impact` edges outward (dependents) and inward (dependencies); relationship type determines whether it "propagates" (e.g., a public signature change propagates through `calls` edges broadly; a private body-only change propagates only if `semantic-impact` links exist to a documented concept).
3. **Topological ordering** (Kahn's algorithm) of the affected-component set, so leaf components are re-verified/re-documented before the higher-level modules/README that summarize them.
4. **Selective regeneration** — only affected `ClaimEntity`/`EvidenceEntity`/`DocEntity` nodes are regenerated and re-verified; everything else in the trust report is carried over unchanged with its prior confidence score and evidence trail intact.
5. **Validation pass** — check cross-references between regenerated and carried-over docs still resolve.

This is a direct adaptation of RepoDoc's incremental mechanism, but extended: RepoDoc only propagates for *documentation* regeneration, you also need to propagate for *claim re-verification* — a changed test file should re-trigger verification of every claim whose evidence included that test, even if the claim's source code didn't change.

8. Scalability

- **Never flood context** — evidence retrieval is graph-query-bounded (k-hop neighborhood + token budget T), not "pull everything from 8 sources." This is the single fix that avoided CodeWiki's 5.3K-token/doc and RepoAgent's 18,629-second/repo blowups in the comparable system.
- **Parallelism** — component-level claim generation and verification run in a thread/process pool respecting the dependency DAG (leaf-first), same pattern as bottom-up doc generation in RepoDoc.
- **Graph store** — use a real graph database (Neo4j or equivalent) rather than an in-memory structure once repos exceed ~50K LOC, so k-hop queries stay sub-second.
- **Incremental cost dominates at scale:** for a mature repo, the overwhelming majority of runs are old-repo/delta runs, not new-repo/full runs — the incremental path is where your latency/cost budget actually matters, not the one-time full build. Design and benchmark for that case first.

- **Sharding for monorepos:** partition the RepoKG by top-level module/package boundary; cross-shard edges (rare, e.g., a shared utils import) are handled by a thin cross-shard index rather than a single global graph, so verification of one package doesn't require loading the whole monorepo's graph.
 - **Caching evidence, not just claims:** test-execution results and CI logs are expensive to regenerate; cache them keyed by commit hash + test file hash so re-verification after an unrelated change doesn't re-run the test suite.
-

9. Function-Level Breakdown (for your methods/implementation section)

Module	Key functions	Purpose
Multi-Source Collector	<code>extract_source()</code> , <code>parse_readme()</code> , <code>extract_tests()</code> , <code>mine_commits(since=)</code> , <code>fetch_issues_prs()</code>	Pull raw evidence per source type; <code>since=</code> param enables delta mode
RepoKG Construction	<code>extract_code_entities()</code> (AST/tree-sitter), <code>extract_structural_edges()</code> , <code>enrich_semantic_edges()</code> (LLM)	Build/patch the graph; structural pass is LLM-free and cheap
Module Clustering	<code>cluster_top_level(K=5)</code> , <code>recursive_subcluster(K=3, T=4096)</code>	Functional-cohesion grouping, not file-size grouping
Claim Generation Agents	<code>scout_agent.identify_regions()</code> , <code>structure_agent.map_layout()</code> , <code>business_rule_agent.extract_rules()</code> , <code>claim_agent.generate(entity)</code>	Produce atomic, falsifiable claims per code entity
Evidence Retrieval	<code>query_khop(entity, k, token_budget)</code> , <code>retrieve_tests_for(entity)</code> , <code>retrieve_commits_for(entity)</code>	Bounded, graph-query-driven retrieval (not flat dump)
Evidence Fusion	<code>fusion_score(claim, evidence[[]])</code> , <code>update_reliability_prior(source_type, outcome)</code>	§4 formula; adaptive weight update
Verification	<code>static_entailment(claim, evidence)</code> , <code>structural_check(claim, ast)</code> , <code>run_tests(claim)</code> , <code>cross_source_consistency(claim, evidence[[]])</code>	Decomposed sub-checks, not one opaque judgment
Calibration	<code>featurize(claim)</code> , <code>calibrated_prob(features)</code> , <code>should_escalate(prob)</code>	§5 — actual calibrated model, ECE-tracked
Completeness	<code>coverage(category)</code> , <code>queue_missing_for_generation()</code>	§6 — closes loop with claim generation
Impact Propagation	<code>detect_changes(diff)</code> , <code>propagate(changed_entities, graph)</code> (BFS from	§7 — the missing incremental path

Module	Key functions	Purpose
	Algorithm 1 style), <code>topo_sort(affected)</code> , <code>selective_regenerate()</code>	
Trust Engine	<code>build_trust_record(claim)</code> , <code>render_conflict_report()</code> , <code>repo_trust_score()</code>	Final explainable output

10. What to Claim as the Novel Contribution

For the publication, the defensible novelty is **not** "a system that documents repositories" (RepoAgent/CodeWiki/RepoDoc already do this well) — it's:

- 1. A **calibrated, per-claim confidence model** for repository documentation, evaluated with ECE, not just an LLM-as-judge quality score (TQS-style metrics measure fluency/structure, not correctness-calibration — a real gap in RepoDocBench's own metric suite).
- 2. **Evidence-typed adaptive fusion** across heterogeneous repo evidence (code/tests/commits/issues/CI), with reliability priors that update per-repository — none of the compared systems weight evidence sources at all.
- 3. **Claim-level incremental re-verification** (not just doc regeneration) tied to Semantic Impact Propagation — i.e., extending an existing mechanism (propagation) to a problem it hasn't been applied to (verification/trust maintenance), which is a legitimate, citable extension rather than a reinvention.

11. Suggested Evaluation Design (reviewers will ask for this)

- Reuse or extend **RepoDocBench**-style repos (24 repos, 8 languages, 3 scale tiers) so your numbers are comparable to the closest published baseline.
- Report **Coverage / Completeness@K** (comparable to RepoDoc) *plus* **ECE and Brier score** for your calibration claims (not reported by any compared system — this is your differentiation).
- Report **Update Recall** and time/token cost for the incremental path (directly comparable to RepoDoc Table 5).
- Construct a **contradiction benchmark**: inject known-false claims (e.g., mismatched docstring vs. implementation) and measure detection precision/recall — this directly tests the "Contradicted" branch, which nothing in prior work benchmarks.

Key sources referenced

- Xu, D. et al. "RepoDoc: A Knowledge Graph-Based Framework to Automatic Documentation Generation and Incremental Updates." arXiv:2604.26523 (2026).
- Luo, Q. et al. "RepoAgent: An LLM-Powered Open-Source Framework for Repository-Level Code Documentation Generation." arXiv:2402.16667 (2024).

- Makharev, V., Ivanov, V. "CodeWiki: Evaluating AI's Ability to Generate Holistic Documentation for Large-Scale Codebases." arXiv:2510.24428 (2025).
- Ouyang, S. et al. "RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph." arXiv:2410.14684 (2024), ICLR 2025.
- Min, S. et al. "FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation." EMNLP 2023.
- "Atomic Calibration of LLMs in Long-Form Generations." arXiv:2410.13246 (2024).
- "VERDI: Single-Call Confidence Estimation for Verification-Based LLM Judges via Decomposed Inference." arXiv:2605.11334 (2026).
- "Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement." ICLR 2025.